A* Maze Solver

By:Omar Ali
Yaseen Mahrous
Mohamed Abdelhamed

SolvingMaze.py

01 Import functions

02 def go_straight():

03 def turn_around():

04 def go_left():

05 def go_right():

Import functions

```
1 from Freenove_Micro_Rover import *
2 from Maze_Solving_Algorithm import *
3 from Maze import *
```

- Import functions of the code
- Imports our A* Algorithm file
- Imports maze
- Imports Rover Control class

Rover initialization:

```
12
13 Rover = Micro_Rover()
14 display.show(Image.HAPPY)
```

- Initializes the rover and imports all functions needed from MicroRover class
- displays :) to show that the rover is ready

def go_straight

```
16
17 def go_straight():
18     Rover.all_led_show(0, 100, 0, 0)
19     Rover.motor(50, 50)
20     sleep(700)
21     Rover.motor(0, 0)
22
```

- Moves rover forward
- Both motors are turned on for 0.7 seconds at speed 50
- Stops motors (speed 0)

def turn_around()

```
def turn_around():
    Rover.motor(60, -60)
    sleep(1300)
```

- Rotates 180 degrees
- One motor turns at speed 60 and the other 60 which rotates
- For 1.3 seconds

def go_left()

```
def go_left():
    Rover.all_led_show(0, 100, 0, 0)
    Rover.motor(80, 80)
    sleep(500)
    Rover.motor(-60, 60)
    sleep(620)
    while (
        pin14.read_digital() == 0
        and pin15.read_digital() == 0
        and pin16.read_digital() == 0
    ):
        Rover.motor(-60, 60)
    Rover.motor(0, 0)
```

- Turns on green LEDs
- Move forward to clear the corner
- Spins motors to the left(Anti clockwise rotation)
- Rotates for 620 ms
- Pins 14, 15, and 16 are the line following sensors beneath the rover.
- These pins return a value of 0 or 1 (No line/white or Line/black)
- While loop: As long as the sensors read 0, it will keep turning

def go_right()

```
def go_right():
    Rover.all_led_show(0, 100, 0, 0)
    Rover.motor(80, 80)
    sleep(500)
    Rover.motor(60, -60)
    sleep(620)
    while (
        pin14.read_digital() == 0
        and pin15.read_digital() == 0
        and pin16.read_digital() == 0
    ):
        Rover.motor(60, -60)
    Rover.motor(0, 0)
```

- Turns on green LEDs
- Move forward to clear the corner
- Spins motors to the right(clockwise rotation)
- Rotates for 620 ms
- Pins 14, 15, and 16 are the line following sensors beneath the rover.
- These pins return a value of 0 or 1 (No line/white or Line/black)
- While loop: As long as the sensors read 0, it will keep turning

A* Algorithm

A* Algorithm

```
60 shortest_path = a_star_search(maze)
61
62 if shortest_path is None:
63     display.show(Image.SAD)
64     raise ValueError("No valid path found in the maze!")
```

- Calls our A* path finding function from Maze_Solving_Algorithm and passes the maze grid to it
- This function Looks at the ‘S’ ‘O’ ‘X’ and ‘E’ and finds the best path from S to E while avoiding the walls (X)
- When it runs it gives us a list of coordinates (0,0) (0,1) (2,0) (2,1) … (9,9)

- If shortest path is not found, meaning that something is missing from the maze or there isn’t a path that the rover can follow
- Shows :(and stops the program with the error No valid path found in the maze.

Coordinates to rover commands

```
def convert_path_to_commands(path):
    commands = []
    directions = {(0, 1): "R", (1, 0): "D", (0, -1): "L", (-1, 0): "U"}
    movement_sequence = [
        directions[(path[i + 1][0] - path[i][0], path[i + 1][1] - path[i][1])]
        for i in range(len(path) - 1)
    ]
    direction_order = "RDLU"
    current_dir = "R"
```

- This function translates the coordinates list into commands for the rover
- Creates empty list of commands that holds instructions for the rover as in (S,L,R)
- In order for the rover to label each step it takes, it needs to have a set translation between movement between 2 points (R,D,L,U)

- Movement_sequence: loop that goes through every coordinate pair in the path to check how you're moving
- We assume the rover is facing right at the beginning and the first move is compared to that
- RDLU is like a compass for the rover

Directional compass and starting direction

```
for move in movement_sequence:
    if move == current_dir:
        commands.append("S")
    else:
        current_idx, move_idx = direction_order.index(
            current_dir
        ), direction_order.index(move)
        commands.append("R" if (move_idx - current_idx) % 4 == 1 else "L")
        current_dir = move
return "".join(commands)
```

- If robot is facing right way then go Straight
- Using the mod operator keeps the value from a range of 1-3 which correspond to the direction in the compass
- For ex: you want to go from right to up then (1 - 0) % 4 = 1 → RIGHT TURN
- For ex: you want to go from right to up then (3 - 0) % 4 = 3 → Left turn
- RDLU

Facing → Going	Indexes	(move-current) % 4	Turn
R → D	0 → 1	(1 - 0) % 4 = 1	R
R → L	0 → 2	(2 - 0) % 4 = 2	L
R → U	0 → 3	(3 - 0) % 4 = 3	L
R → R	0 → 0	(0 - 0) % 4 = 0	S

Sensor readings and rover allignment

```
commands += 'S' # For completing the last command
```

- Adds the last move needed to reach the E

```
while commands:
    sensor_value = (
        pin14.read_digital() << 2 | pin15.read_digital() << 1 | pin16.read_digital()
    )

    if commands and sensor_value == 7:
        current_command, commands = commands[0], commands[1:]
        if current_command == "S":
            go_straight()
        elif current_command == "R":
            go_right()
        elif current_command == "L":
            go_left()
    elif sensor_value in [2, 5]:
        Rover.motor(70, 70)
    elif sensor_value in [4, 6]:
        Rover.motor(20, 90)
    elif sensor_value in [1, 3]:
        Rover.motor(90, 20)
    elif sensor_value == 0:
        turn_around()
```

- If the sensor is aligned then it removes the first command from the string
- The sensors (pins 14,15,16) give back values (0 or 1) and using bitshifting we can combine them into one 3 bit number.
- This allows us to fix the rovers path using if statements according to the value we get after bitshifting the sensor values

Bitshifting to use IF statements

pin14	pin15	pin16	Value	Meaning
1	1	1	7	All sensors see line (centered)
0	1	0	2	Only center sees line
1	0	0	4	Only left sensor sees line
0	0	0	0	All sensors off line (lost)

- Sensor value [2,5]: Rover is almost on track, move forward slowly
- Sensor value [4,6]: Left sensor sees line, rover drifting to the right so curve to the left
- Sensor value [1, 3]: Right sensor sees line, rover drifting to the left so curve to the right

- We treat the sensor readings like 3 bits:
- [pin14] [pin15] [pin16] → [left][middle][right]
  ↑ ↑ ↑
  ×4 ×2 ×1
- << bit shift/move number to the left in binary
- Pin 16 is kept as is, Pin 15 is multiplied by 2, and Pin 14 is multiplied by 4
- We add them using an OR gate |
- If all sensors are on: 4 +2 +1 = 7 (perfectly aligned)
- If only the center is on : 0+2+0= 2
- If only left sensor is on : 4+0+0=4
- If nothing is on: 0+0+0=0
- Right sensor can be taken directly without bitshifting since it directly corresponds to the value of pin 6 so it's either 1 or 0

Maze Finished:

```
Rover.motor(0, 0)
Rover.all_led_show(100, 0, 0, 0)
display.show(Image.HEART)
```

- Stop all motors
- Turn LEDs red
- Display heart icon

Thank you!